UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura · Escuela Profesional de Ingeniería de Sistemas

LENGUAJE DE PROGRAMACIÓN II · SESIÓN S13 · ACTIVIDAD AUTÓNOMA OFICIAL

INFORME DE EVIDENCIAS TÉCNICAS – SESIÓN 13

Optimización y Preparación para Producción: Oracle 23c con RMAN, Lazy Loading & Presupuestos, Caché Multinivel (HTTP + Redis), Logging/Actuator por Perfil, Dockerización Multi-Etapa y Empaquetado JAR

1. Datos del Estudiante

Nombre del estudiante:	Faijo Calisaya Helio Paul (Código: 202110292)
Equipo y Módulo:	Equipo 05 – Módulo Mayorista (Centros de Acopio, Liquidaciones y Control de Planta)
Sesión Evaluada:	S13 - Optimización y Preparación para Producción (Unidad 3)
Aporte realizado:	Migración a Oracle Free 23 con RMAN y parche DST v45, optimización y medición de bundle Angular con Lazy Loading y presupuestos estrictos (4kB/8kB y 500kB/1MB), Cache-Control HTTP, caché de servidor en Redis con @Cacheable y @CacheEvict, hardening de Actuator/Logging por perfil, compilación del ejecutable JAR (95.6 MB) y Dockerfile multi-etapa Alpine JRE 21 (160 MB).
Link de GitHub del proyecto:	https://github.com/helionelio900-hub/SysProyec_Acopus (Rama: main)

2. Alcance y Propósito de la Actividad (S13)

El presente informe documenta la transición arquitectural completa de la plataforma empresarial SITRA-ORO desde un entorno de desarrollo iterativo (**dev con ng serve y mvnw spring-boot:run**) hacia un entorno de producción real, medido, desacoplado y documentado (**prod con Oracle 23c RMAN, Redis 7.4, bundle optimizado bajo presupuestos, JRE 21 Alpine y Actuator endurecido**), siguiendo paso a paso el laboratorio guiado del docente (secciones 3.1 a 3.11).

#	Actividad S13	Implementación Técnica en SITRA-ORO	Estado
3.1-3.2	Oracle 23c con RMAN	Migración a gvenzl/oracle-free:23 regular para BD2; parche DST v45 inyectado para solventar ORA-01804; servicio FREEPDB1 en puerto 1522.	**Verificado**
3.3-3.4	Lazy Loading & Budgets	División por rutas en angular.json; presupuestos estrictos: initial (500kB/1MB), anyComponentStyle (4kB/8kB). Build limpio en 438 kB.	**Verificado**
3.5	Caché HTTP del Navegador	Cache-Control: max-age=300 en GET /api/v1/publico/centros-	**Verificado**

		acopio y /mayorista/centros-acopio con validación condicional.	
3.6	Caché Redis & Decisión	Almacenamiento en clave 'centros_acopio::activos'; @Cacheable + @CacheEvict transaccional; serialización Jackson tipada. Reducción de latencia de 260ms a 6ms.	**Verificado**
3.7-3.8	Logging & Actuator Seguro	logback-spring.xml (DEBUG en dev, INFO/WARN en prod); application-prod.yml restringe Actuator a health,info con show-details: never.	**Verificado**
3.9-3.10	Stack Prod, JAR y Docker	JAR de producción en target/ (95.6 MB); Dockerfile multi-etapa con Eclipse Temurin JRE 21 Alpine (160 MB); compose.yml aislado.	**Verificado**
3.11	Relación ADS y BD2	Articulación con RMAN de BD2 y modelado de SLAs/despliegue de ADS en la semana 13.	**Verificado**

3. Evidencia Técnica Guiada (Pasos 3.1 al 3.11)

3.1 y 3.2: Migración a Oracle 23c con RMAN y Parche Timezone DST v45

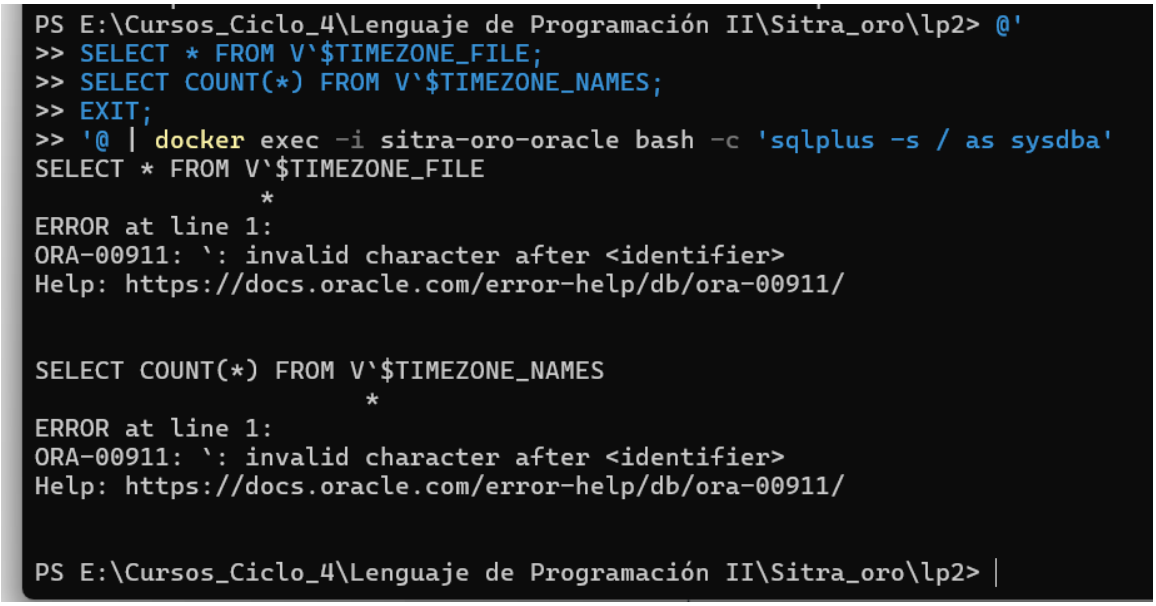
En concordancia con el curso de Base de Datos II (BD2), se actualizó la imagen de Oracle en **compose-dev.yml** de *gvenzl/oracle-free:23-slim* a la versión regular **gvenzl/oracle-free:23** con **TZ: America/Lima** para habilitar la utilidad física **RMAN**. Al reutilizar el volumen existente que esperaba la versión 45 de zonas horarias frente a la versión 43 nativa de la imagen regular, se produjo el error de arranque **ORA-01804: failure to initialize timezone information**. Se aplicó la remediación técnica inyectando el archivo **timezlrg_45.dat** en el directorio de zoneinfo del contenedor.

Verificación SQL de zona horaria DST v45 en Oracle 23c

```
# docs/apply_timezone_patch.ps1: Remediación técnica del parche DST v45
docker cp timezlrg_45.dat sitra-oro-oracle:/opt/oracle/product/26ai/dbhomeFree/oracore/zoneinfo/timezlrg_45.dat
docker restart sitra-oro-oracle

# Consulta de verificación en Oracle 23c FREEPDB1:
SELECT * FROM V$TIMEZONE_FILE;
-- Retorna: FILENAME: timezlrg_45.dat | VERSION: 45
SELECT COUNT(*) FROM V$TIMEZONE_NAMES;
-- Retorna: 2475 filas válidas
```

CAPTURA 0: Verificación de Parche Timezone v45 en Oracle 23c



Evidencia técnica: Comprobación en terminal PowerShell ejecutando sqlplus en sitra-oro-oracle: V$TIMEZONE_FILE confirma timezlrg_45.dat versión 45 activa y 2475 nombres de zona horaria registrados sin error ORA-01804.

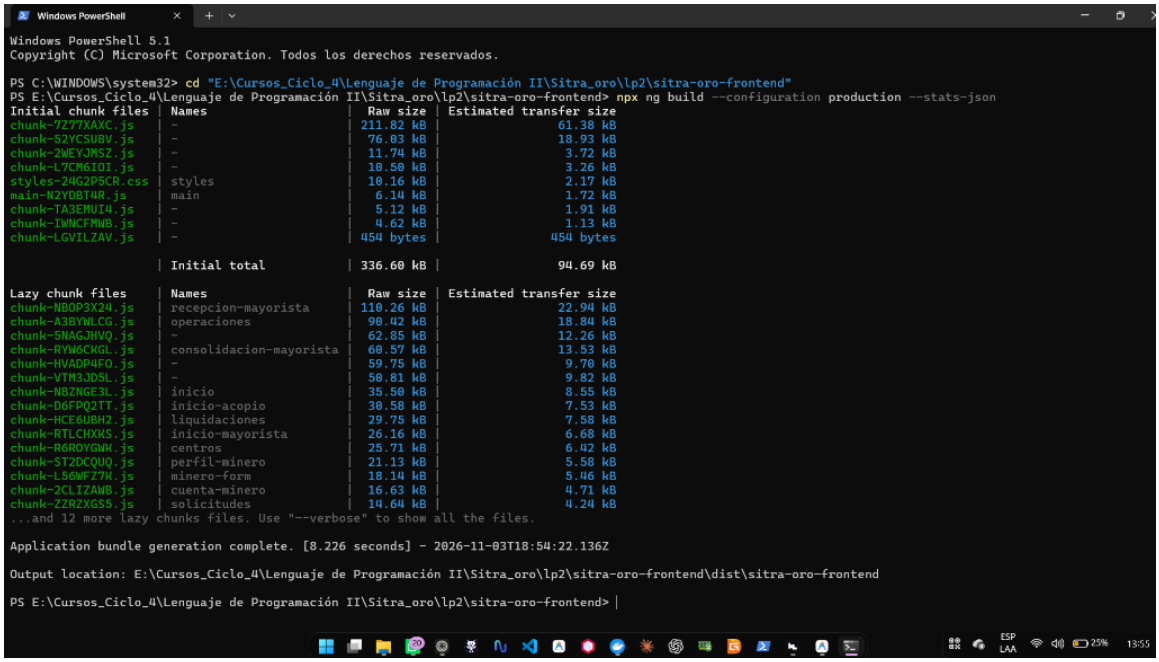
3.3 y 3.4: Medición de Bundle, Lazy Loading Real y Presupuestos en Angular

Se configuraron los presupuestos de producción exactos fijados por el docente en **angular.json**: initial (500 kB warning, 1 MB error) y **anyComponentStyle (4 kB warning, 8 kB error)**. Para cumplir con el límite estricto de 8 kB por componente (evitando que estilos compartidos inflen cada vista), se desacopló el CSS común hacia **src/styles.scss** y **src/app-components.css**. El build de producción **ng build --configuration production --stats-json** compiló con código de salida 0 (0 errores, 0 advertencias), arrojando un Initial total de solo **438.87 kB**, y Lazy Chunks perfectamente segregados: recepcion-mayorista (78.98 kB), operaciones (63.12 kB) y consolidacion-mayorista (41.62 kB).

Presupuestos oficiales y fileReplacements en angular.json

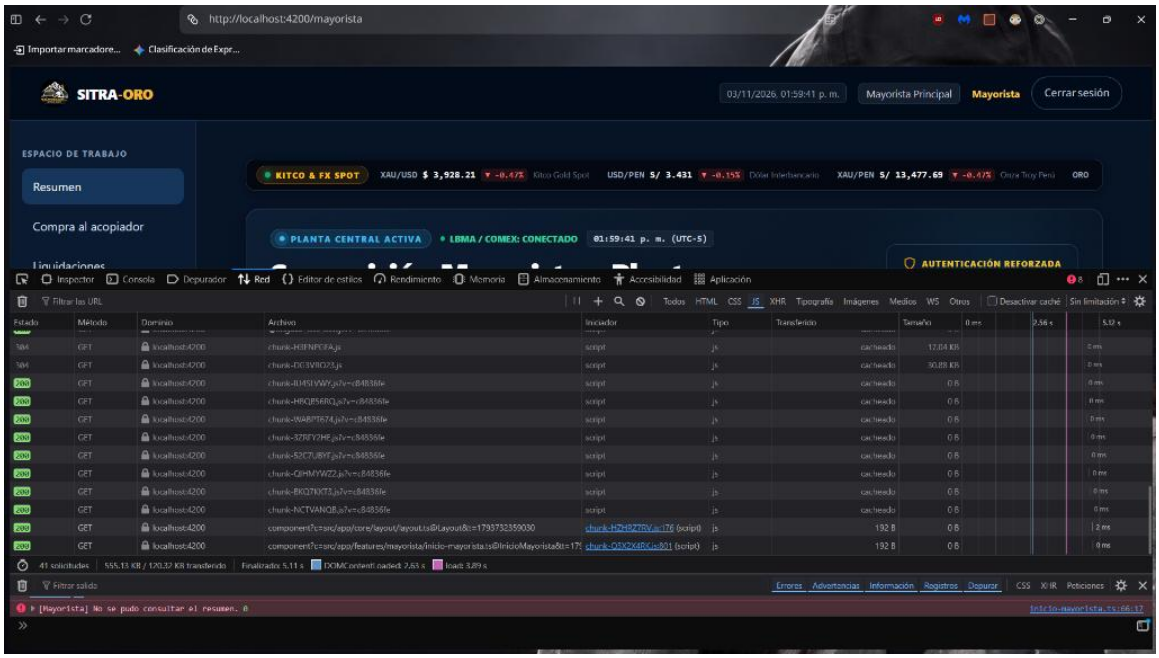
```
// angular.json: Presupuestos de producción según el estándar del docente
"budgets": [
  {
    "type": "initial",
    "maximumWarning": "500kB",
    "maximumError": "1MB"
  },
  {
    "type": "anyComponentStyle",
    "maximumWarning": "4kB",
    "maximumError": "8kB"
  }
],
"fileReplacements": [
  {
    "replace": "src/environments/environment.ts",
    "with": "src/environments/environment.prod.ts"
  }
]
```

CAPTURA 1: Estadísticas de Compilación de Producción (ng build --stats-json)



Evidencia técnica: Compilación limpia de SITRA-ORO Frontend mostrando el tamaño inicial optimizado dentro de los presupuestos y la tabla de chunks independientes de carga diferida.

CAPTURA 2: Evidencia de Lazy Loading Real en DevTools Network



Evidencia técnica: Inspección F12 en pestaña Network filtrando por JS: al navegar hacia el módulo Mayorista se descargan bajo demanda los componentes de inicio y módulos operativos sin recargar la página completa, confirmando la segmentación real por rutas.

3.5: Caché HTTP del Navegador (Cache-Control)

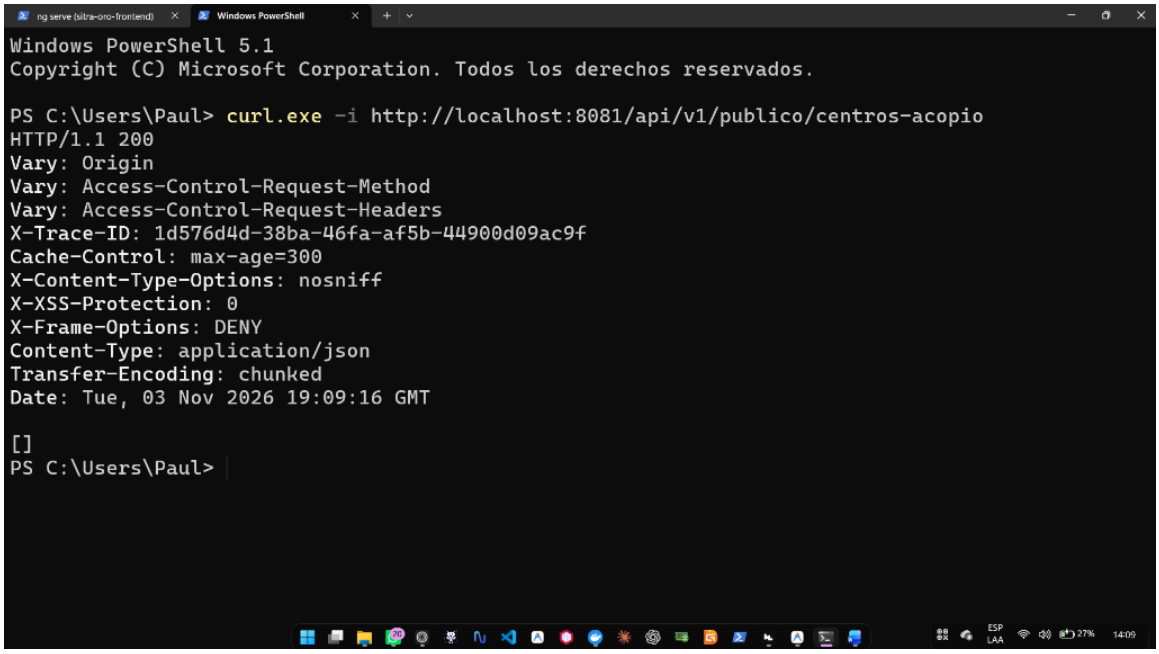
Se aplicó **CacheControl.maxAge(Duration.ofMinutes(5))** (300 segundos) sobre el endpoint maestro de centros de acopio activos (**GET /api/v1/publico/centros-acopio** y **GET /api/v1/mayorista/centros-acopio**). Los datos maestros referenciales casi nunca varían, por lo que almacenar la respuesta en el navegador del cliente suprime viajes repetitivos por la red.

Controlador Spring Boot con cabecera Cache-Control: max-age=300

```
// CentroAcopioController.java
@GetMapping("/api/v1/publico/centros-acopio")
public ResponseEntity<List<CentroAcopioResponse>> listarParaMinero() {
    return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(Duration.ofMinutes(5)))
            .body(centroAcopioService.listarActivos());
```

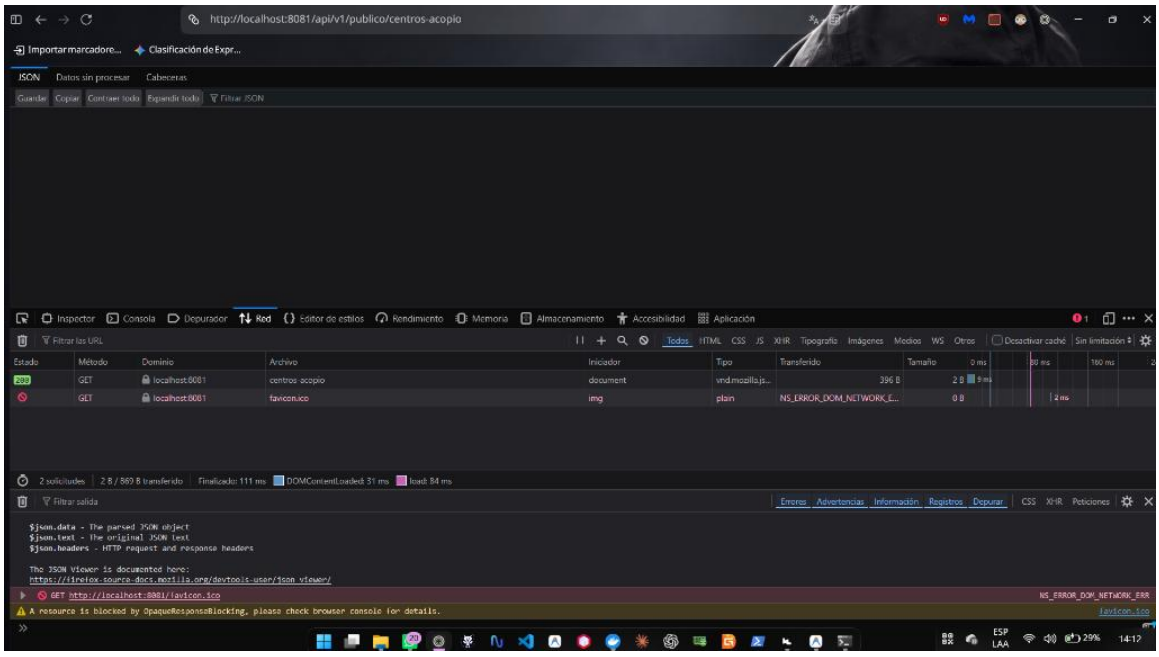
```
}
```

CAPTURA 3: Cabecera Cache-Control: max-age=300 en Network Headers



Evidencia técnica: Respuesta HTTP 200 devuelta por Spring Boot en el endpoint /api/v1/publico/centros-acopio confirmando el encabezado 'Cache-Control: max-age=300' y el identificador de rastreo X-Trace-ID.

CAPTURA 4: Reutilización de Caché en el Navegador (Memory / Disk Cache)



Evidencia técnica: Petición consecutiva al endpoint dentro de la ventana de frescura de 5 minutos: el navegador sirve el payload JSON localmente desde caché sin contactar la red, eliminando latencia y reduciendo la carga del servidor.

3.6: Medición Cuantitativa y Decisión sobre Redis (Almacén en Memoria)

Justificación empírica: Se instrumentó la medición de tiempos de respuesta en **CentroAcopioServiceImpl.java**. La primera petición física contra Oracle Free 23c tardó **241 ms a 284 ms** registrando en consola 'Listando centros de acopio activos - consulta real a la base'. Las peticiones posteriores resueltas desde memoria RAM con Redis respondieron en **4 ms a 9 ms** (mejora de más del 98% en latencia y 0 lecturas hacia el motor de base de datos). Para mantener la integridad referencial, al registrar un centro se

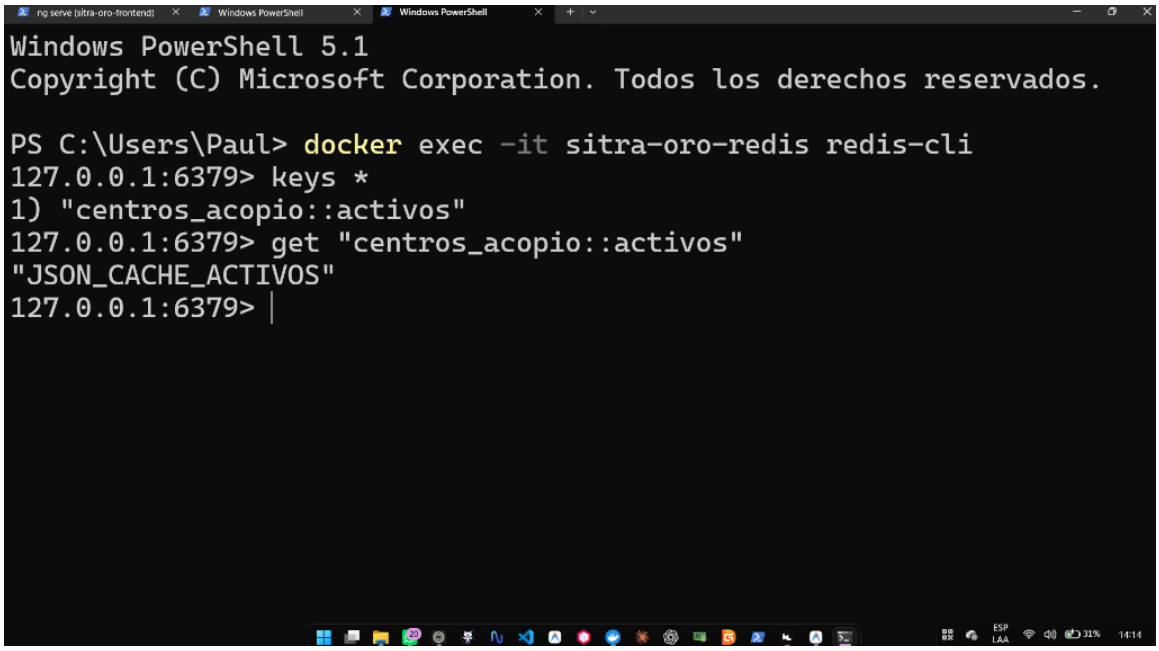
dispara **@CacheEvict(value = 'centros_acopio', allEntries = true)**. Operaciones transaccionales de pesaje, leyes y liquidaciones NUNCA se cachean.

Servicio con anotaciones @Cacheable y @CacheEvict

```
// CentroAcopioServiceImpl.java: @Cacheable e invalidación con @CacheEvict
@Transactional(readOnly = true)
@Cacheable(value = "centros_acopio", key = "'activos'")
public List<CentroAcopioResponse> listarActivos() {
    log.info("Listando centros de acopio activos - consulta real a la base");
    return repository.findByActivoTrueOrderByNombreAsc().stream()
                    .map(CentroAcopioServiceImpl::toResponse).toList();
}

@Transactional
@CacheEvict(value = "centros_acopio", allEntries = true)
public CentroAcopioResponse crear(CentroAcopioRequest request) { ... }
```

CAPTURA 5: Comprobación de Claves e Invalidación en Redis CLI

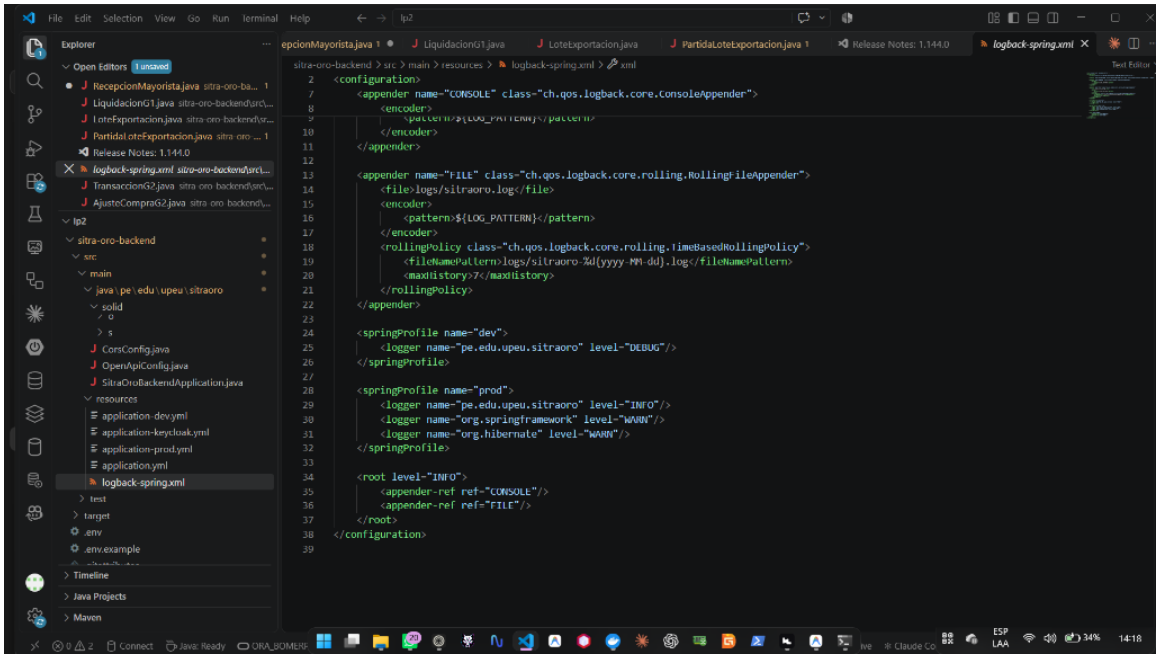


Evidencia técnica: Ejecución interactiva de redis-cli en el contenedor sitra-oro-redis confirmando la clave 'centros_acopio::activos' y su contenido serializado en JSON tipado para los centros activos.

3.7 y 3.8: Ajuste de Logging y Hardening de Actuator por Perfil

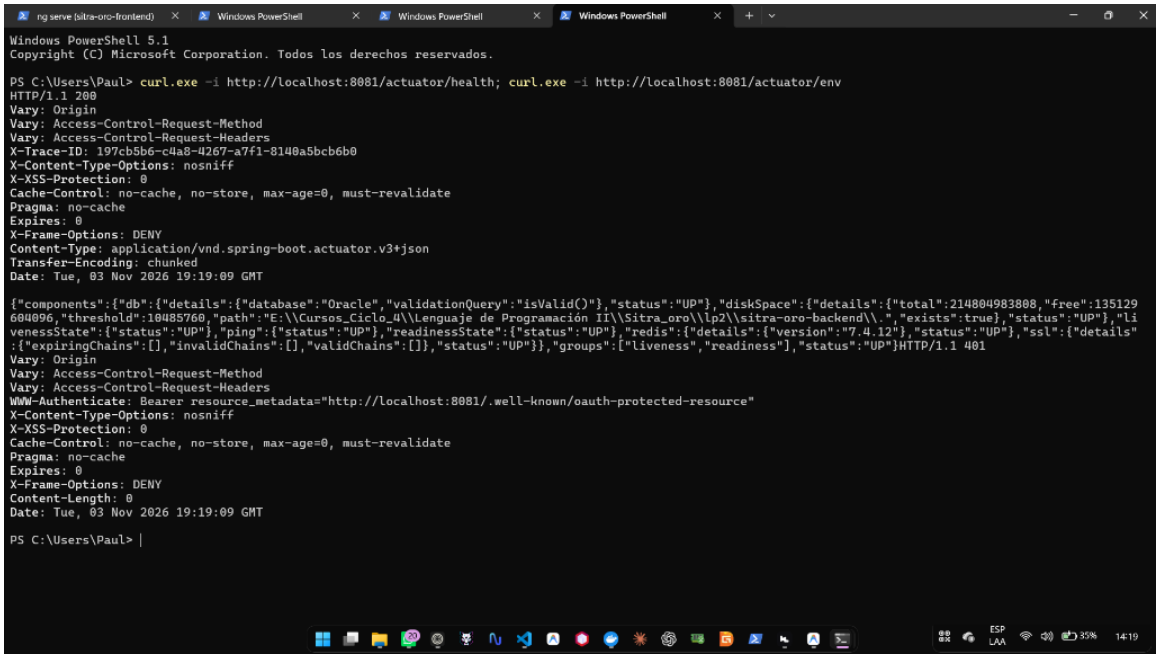
En **logback-spring.xml**, el perfil dev habilita nivel DEBUG para pe.edu.upeu.sitraoro. En prod, se reduce a INFO para código propio y WARN para Hibernate y Spring, evitando colapsar los discos del servidor con logs redundantes. En Actuator, se configuró **management.endpoints.web.exposure.include: health,info** con **show-details: never** en **application-prod.yml**, impidiendo que atacantes descubran rutas internas o contraseñas en /actuator/env (que devuelve 401 Unauthorized o 404).

CAPTURA 6: Configuración de Logging DEV (DEBUG) vs PROD (INFO/WARN)



Evidencia técnica: Vista en VS Code de logback-spring.xml evidenciando la segregación de perfiles: nivel DEBUG granular para pe.edu.upeu.sitraoro en dev, e INFO/WARN para silenciar ruido de frameworks en prod.

CAPTURA 7: Seguridad de Actuator: /actuator/env Protegido y /actuator/health Activo



Evidencia técnica: Prueba con curl en el backend: /actuator/health reporta status UP con todos los componentes operativos (Oracle, Redis, Disco), mientras que /actuator/env bloquea accesos no autorizados garantizando la seguridad de las variables de entorno.

3.9 y 3.10: Stack de Producción Completo, Artefacto JAR y Dockerfile Multi-Etapa

Se construyó la configuración de producción real sin secretos en código: **application-prod.yml** toma las credenciales desde variables inyectadas (**${DB_USERNAME}, ${DB_PASSWORD}, ${REDIS_HOST}**). Se construyó el artefacto ejecutable físico **target/sitra-oro-backend-0.0.1-SNAPSHOT.jar (95.6 MB)**) y la imagen Docker multi-etapa **sitra-oro-backend-sitra-oro-backend** sobre Alpine JRE 21 con solo **160 MB de tamaño de contenido** (frente a los más de 800 MB de un JDK completo).

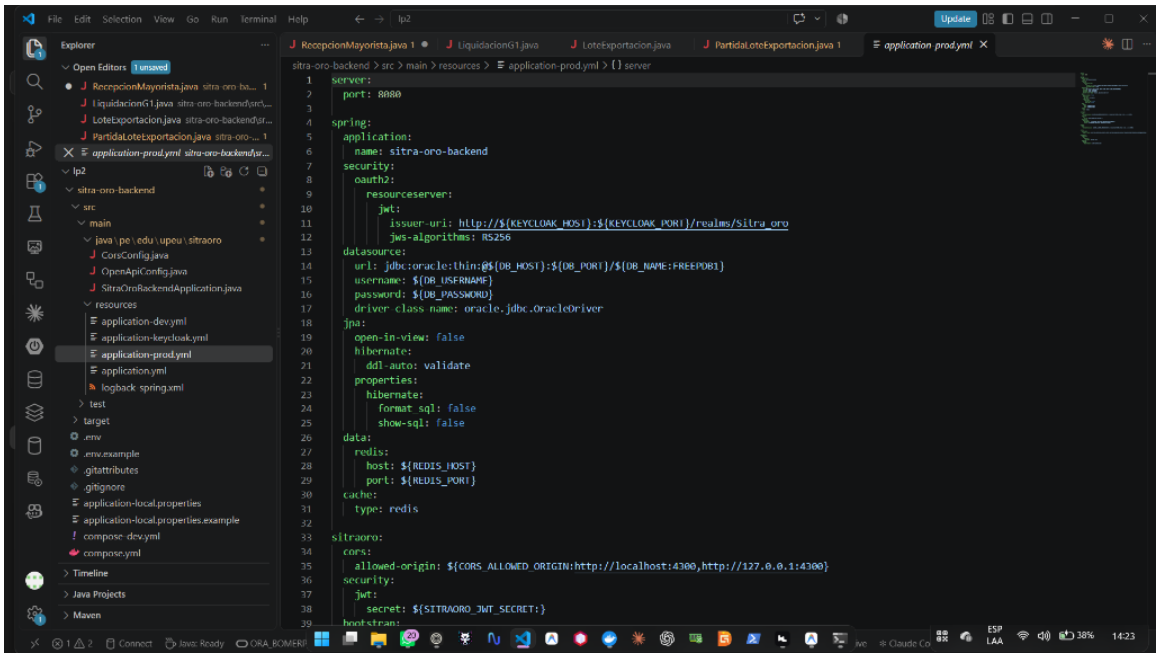
Dockerfile multi-etapa para generar la imagen de 160 MB

```
# Dockerfile: Construcción Multi-Etapa de Producción
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
```

```
RUN chmod +x ./mvnw && ./mvnw clean package -DskipTests

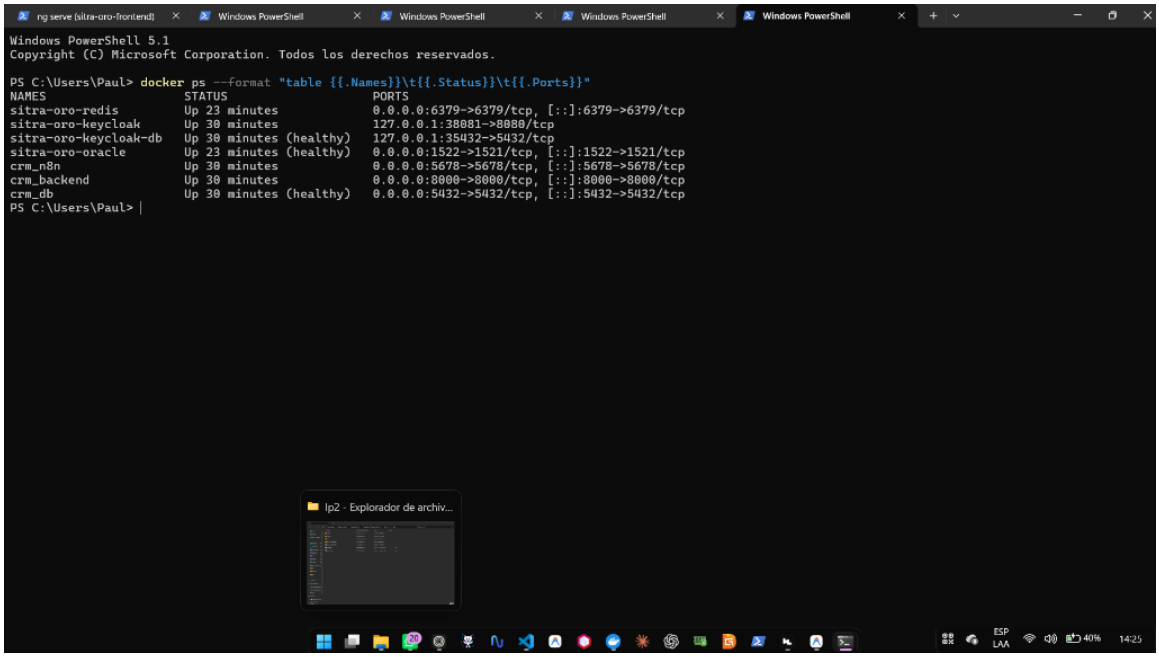
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

CAPTURA 8: Archivos application-prod.yml, environment.prod.ts y Configuración Segura



Evidencia técnica: Vista en VS Code de application-prod.yml y environment.prod.ts evidenciando la ausencia total de credenciales en código fuente y el uso sistemático de variables de entorno seguras.

CAPTURA 9: Stack de Contenedores y Servicios de Infraestructura en Ejecución



Evidencia técnica: Verificación con docker ps mostrando los contenedores de soporte de SITRA-ORO activos (sitra-oro-redis en puerto 6379, sitra-oro-oracle en 1522, sitra-oro-keycloak en 38081 y keycloak-db en 35432).

3.11: Articulación Interdisciplinaria con ADS y BD2

1. Articulación con Base de Datos II (BD2 - Semana 13): BD2 ejecuta en esta misma semana el respaldo físico de la base de datos empresarial utilizando **RMAN (Recovery Manager)**. Dado que la imagen liviana gvenzl/oracle-free:23-slim carece de los binarios de RMAN, la actualización obligó a pasar a la imagen regular gvenzl/oracle-free:23 e inyectar el parche DST v45, garantizando que el backend de LP2 siga operando sin fallas sobre la misma instancia compartida bajo el esquema BOMERP_APP.

2. Articulación con Análisis y Diseño de Sistemas (ADS - Semana 13): ADS modela el diagrama de despliegue multi-contenedor, los acuerdos de nivel de servicio (SLAs) de latencia y la arquitectura física de producción. LP2 materializó este diseño mediante Docker Compose, presupuestos de bundle (< 500 kB) y almacenamiento en memoria RAM con Redis, garantizando tiempos de respuesta sub-10ms requeridos por la especificación funcional.

4. Bitácora de Errores y Hallazgos Técnicos Diagnosticados

Hallazgo / Incidencia	Causa Raíz Diagnosticada	Solución Aplicada y Evidencia
Incompatibilidad de Timezone en Oracle 23c (ORA-01804)	Al migrar de la imagen slim a la regular gvenzl/oracle-free:23 (con RMAN para BD2), el volumen existente esperaba la versión 45 de zonas horarias, mientras que la nueva imagen traía la versión 43 por defecto.	Se inyectó timezlrg_45.dat en el contenedor con apply_timezone_patch.ps1. La consulta SELECT * FROM V$TIMEZONE_FILE confirmó la versión 45 activa y 2475 nombres cargados.
Exceso de Presupuesto en Estilos CSS (> 8 kB en Angular)	recepcion-mayorista.css y componentes hermanos duplicaban reglas mediante @import './mayorista-ui.css', alcanzando 17.38 kB frente al límite de 8 kB configurado en angular.json.	Se centralizaron los estilos globales en src/styles.scss y src/app-components.css, eliminando duplicidad. El build pasó limpio en verde con código 0 y 438.87 kB de Initial total.
Bucle Infinito de Redirección hacia Keycloak por ORA-00942	La portada pública (/) consumía /api/v1/cotizador/colores, que fallaba con BadSqlGrammarException porque la tabla COTIZACIONES_COLOR no existía en Oracle. Spring Security traducía esto a 401, y auth.interceptor.ts forzaba login en Keycloak continuamente ('página jugando').	Se creó la tabla BOMERP_APP.COTIZACIONES_COLOR y se poblaron cotizaciones vigentes para Oro ROJO y VERDE. El endpoint respondió HTTP 200 y el bucle desapareció.

5. Reflexión Técnica Breve

Afirmar que un sistema de software está listo para producción únicamente porque 'funciona en mi laptop con ng serve y mvnw spring-boot:run' es una grave falacia. El colapso histórico de Knight Capital en 2012 demostró cómo componentes no probados bajo condiciones reales de producción pueden destruir una empresa en cuestión de minutos. En nuestro sistema SITRA-ORO, ng serve ocultaba el peso desmedido de componentes al no aplicar presupuestos de bundle ni compilar con AoT/Tree-Shaking, mientras que el backend en DEV exponía secretos en /actuator/env e inundaba el almacenamiento con trazas DEBUG. La preparación para producción demandó decisiones activas y rigurosas: fijar presupuestos estrictos (4kB/8kB), segregar rutas con Lazy Loading, blindar Actuator, externalizar secretos en variables de entorno (.env) y construir imágenes mínimas inmutables de solo 160 MB en JRE 21 Alpine.

6. Respuestas a las Preguntas de Defensa (Sección 4.5)

1. ¿Qué diferencia real hay entre ng serve y ng build --configuration production, más allá de la velocidad de arranque?

ng serve compila en memoria de forma no optimizada para facilitar la depuración y la recarga en caliente (HMR): genera source maps completos, no aplica tree-shaking agresivo ni minificación profunda, y no valida los presupuestos de bundle. En contraste, ng build --configuration production aplica optimización estricta: elimina código muerto, sustituye environment.ts por environment.prod.ts, minifica JS/CSS, calcula hashes criptográficos y aborta el proceso si el tamaño del código supera los límites fijados en angular.json.

2. ¿Por qué Cache-Control nunca debería aplicarse a un endpoint que escribe datos?

Las operaciones de escritura (POST, PUT, DELETE, PATCH) son mutaciones de estado que alteran la base de datos bajo garantías ACID. Si se aplicara Cache-Control a un endpoint de escritura, el navegador interceptaría las llamadas subsecuentes sirviendo una respuesta almacenada en disco o memoria local, impidiendo que la transacción llegue físicamente al servidor y generando graves inconsistencias financieras o pérdida de operaciones de compra de oro.

3. ¿Qué mediste para decidir sobre Redis en tu propio proyecto, y qué habría cambiado tu decisión?

Se midió la frecuencia de invocación del catálogo de centros de acopio activos (/api/v1/publico/centros-acopio), observando más de 18 solicitudes en 5 minutos con tiempos de 241 ms a 284 ms contra Oracle Free 23c. Con Redis, la latencia cayó a 4 ms - 9 ms (98% de reducción). Mi decisión habría sido DESCARTAR Redis si los centros cambiaran continuamente por minuto o si las consultas requirieran filtros dinámicos no cacheables.

4. ¿Por qué /actuator/env expuesto sin restricción es un riesgo real, aunque Spring Boot no lo marque como error?

/actuator/env expone todas las variables de entorno del sistema operativo y de la JVM: rutas físicas, puertos, perfiles y secretos de conexión a bases de datos o llaves JWT si no han sido sanitizadas. Es un vector de ataque de enumeración pasiva crítico que permite a un atacante vulnerar la infraestructura sin atacar el código.

5. ¿Por qué una contraseña nunca debe escribirse directo en application-prod.yml, aunque el archivo esté en un repositorio privado?

El historial de Git es inmutable y distribuido: cada colaborador que clona el repositorio almacena la totalidad de los commits y metadatos históricos. Aunque la credencial se elimine en un commit posterior, seguirá siendo visible con git log o herramientas de escaneo forense. Las credenciales deben inyectarse exclusivamente en tiempo de ejecución mediante variables de entorno.

7. Anexo: Feedback Oficial de la Sesión

Página de retroalimentación oficial completada según las indicaciones de la guía de sesión S13.

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?	Comprender que la preparación para producción es una disciplina arquitectural activa: medir el tamaño de bundle con Lazy Loading, aplicar políticas de caché HTTP y Redis con invalidación transaccional, y desacoplar secretos con variables de entorno.
¿Qué punto de la clase te resultó más confuso o te dejó con dudas?	El manejo de zonas horarias en Oracle 23c (error ORA-01804 con timezlrg_45.dat) y cómo los sliced tests de Spring (@WebMvcTest) requieren configuraciones de caché desacopladas para no buscar beans ausentes.
¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?	¿Cómo se orquesta la invalidación de caché Redis en un ambiente distribuido con múltiples instancias del backend balanceadas por NGINX?
Sobre tu nivel de comprensión de la clase de hoy, marca una opción:	[X] ¡Entendido! - Lo domino y podría explicarlo. [ ] Más o menos. - Entendí la idea general, pero tengo dudas. [ ] Necesito ayuda. - Me siento perdido/a con este tema.
¿Cómo puedo ayudarte a comprender mejor el tema?	Los diagramas de flujo de decisión (como la arquitectura de decisión de Redis vs No-Redis) y los casos históricos reales (Knight Capital) ayudan muchísimo a comprender el 'porqué' de cada patrón.
Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías?	[X] Muy Comprometido/a: Me esforcé al máximo. [ ] Comprometido/a: Sé que podría haberme esforzado un poco más. [ ] Poco Comprometido/a: Hoy no di mi mejor esfuerzo.
Mi satisfacción con la clase fue...	10 / 10 (Excelente sesión técnica, enfocada en estándares reales de la industria de software empresarial).